

CORE RULES

Locals by default	count = 0 · function f()...end
Equality / inequality	a is b · a is not b · a != b
Falsey values	nil, false (0, "", {} are true)
Statement separator	Newline, or ; to disambiguate
Comments	-- line · --[[long --]]
Indentation	Three spaces

```
name = 'Ada' -- implicit local
local count:num = 0 -- explicit local and type
global app_name = 'Demo' -- global
MAX_RETRIES <const> = 3 -- local constant
count += 1 -- also -= *= /= %= .=
count++ -- statement only; no --
a, b = b, a -- multiple assignment / swap
_, value = getResult() -- _ discards, cannot be read
```

The first non-nil assignment fixes a local's type. Assigning nil clears the value, not the type. Use :any only for genuinely unrelated types.

LITERALS AND STRINGS

```
n = 42; hex = 0xff; bits = 0b1010; real = 1.5e-2
ok = true; missing = nil
text = 'single quotes are preferred'
multiline = [[verbatim text
over several lines]]
greeting = f'Hello, {name}! 2 + 3 = {2 + 3}'
braces = f'Use {{ and }} for literal braces'
digits = r'\d+' -- raw regular expression
```

TABLES, ARRAYS AND ACCESS

```
user = { name = 'Ada', ['role-name'] = 'admin' }
items = { 'zero', 'one', 'two' } -- sequence
scores = array<int> { 10, 20, 30 } -- typed array
buffer = array<byte, 1024> -- sized array

user.name = 'Grace'
first = items[0] -- zero-based
count = #scores
user.role = nil -- removes a field

for k, v in user do print(k, v) end
for idx, v in scores do print(idx, value) end
for _, value in scores do print(value) end
for value in values(scores) do print(value) end
```

CONDITIONS AND LOOPS

```
if score >= 90 then
  grade = 'A'
elseif score >= 75 then
  grade = 'B'
else
  grade = 'C'
end

while ready is false do poll() end
repeat
  line = readLine()
until line is 'quit'

for i in {0 to 5} do print(i) end -- 0..4
for i in {1 into 5} do print(i) end -- 1..5
for i in {0 to 1 by 0.25} do print(i) end
for {0 to 3} do retry() end -- no variable
```

```
for key, value in collection do
  if value is nil then continue end
  if key is 'stop' then break end
end
```

FUNCTIONS

```
function clamp(Value:num, Low:num, High:num):num
  if Value < Low then return Low end
  if Value > High then return High end
  return Value
end

function splitPair(Value:str):<str, str>
  return Value.sub(0, 1), Value.sub(1)
end

left, right = splitPair('AB')
double = x => x * 2
add = (a, b) => a + b
work = x => do
  result = prepare(x)
  return process(result)
end
```

Functions are closures. A call in the final position of an assignment, argument list or return forwards all its results; elsewhere it contributes one.

```
function sum(...):num
  total = 0
  for _, value in {...} do total += value end
  return total
end
```

OPERATORS AND COMPACT DECISIONS

+ · * / % **	Arithmetic; ** right-associative
a .. b · #value	Concatenation (numbers coerce) / length
is · is not · != · < <= > >=	Comparison; 0 <= x < 10 chains
not · and · or	Logic, standard falsiness
~ & ^ << >>	Bitwise, C-style
flags has MASK	True when (flags & MASK) != 0
value in container	Range, substring, array value, table key
cond ? yes : no	Ternary, standard falsiness
value ?? yes : no	Extended ternary; 0 '' {} also falsey
a ≈ b	Approximate; tolerance 1e-5

NIL SAFETY AND DEFAULTS

??, ??= and ?! use **extended falsiness** (nil false 0 '' and empty collections). Safe navigation and ?= test **only nil**.

```
label = supplied ?? 'Untitled' -- any empty
timeout ?= 30 -- only when nil
label ??= 'Untitled' -- when empty
supplied ?! return ERR_InvalidInput -- guard
if supplied?? then use(supplied) end -- postfix test
```

```
city = user?.profile?.address?.city
item = data?[index] -- nil-safe index
name = user?.name ?? 'Guest'
user?.stats?.count += 1 -- RHS skipped too
```

RANGES, SLICING AND MEMBERSHIP

```
exclusive = {0 to 5} -- stop excluded
inclusive = {0 into 5} -- stop included
stepped = {0 to 10 by 2}

word = 'Hello, World!'[{7 to 12}] -- 'World'
tail = items[{-3 into -1}] -- last three
reverse = items[{-1 into 0}] -- reversed
inside = value in {0 to 10}
```

Ranges iterate and slice strings, sequence tables and arrays. Negative indices count from the end (-1 is last).

TYPES AND CONTRACTS

```
local total:num = 0
local title:str = 'Report'
local enabled:bool = true
local options:table = {}
local callback:func
local rows:array<table> = array<table> {}
local variants:any = nil
```

```
function parse(Text:str!):<num, str!> -- ! rejects nil
    return #Text, Text
end
```

```
if value is <num> then print('number') end
if value != <array int> then raise 'Expected ints' end
```

Native annotations: any nil bool num str table func obj range userdata struct<Name> array<T>. Array element storage: byte int int64 float double str table obj struct<Name>, nested array<...> or any — array<num> is invalid.

ERROR HANDLING

```
try
    _, data = check file.acRead(buffer)
except e when ERR_NotFound, ERR_Read
    print(f'Read failed: {e.message}')
except e
    print(f'Error {e.code}: {e.message}')
    raise -- rethrow
success
    consume(data)
end
```

```
raise 'Invalid input' -- ERR_Exception
raise ERR_Args
raise ERR_Args, 'Expected a path'
value = ok ? result : raise(ERR_Failed, 'No result')
```

try catches exceptions; it does not check returned ERR values. Use check Expression, or checkall across a block of direct native calls.

```
try
    checkall
        file.acRead(buffer)
        task.acSetKey('Status', 'ready')
    end
except e
    print(e.message)
end
```

DETERMINISTIC CLEANUP

```
handle <close> = acquire() -- __close on exit
```

```
defer
    releaseLock()
end
```

```
defer(saved_path) -- snapshot now
    removeTemporary(saved_path)
end(path)
```

Cleanup runs on normal exit, return, break, continue and error unwinding. In one scope <close> handlers run before defer blocks; each group last-in, first-out.

PATTERN SELECTION

```
label = choose status from
    200 -> 'OK' -- literal
    404 -> 'Not found'
    < 500 -> 'Client error' -- relational
    <num> when status >= 500 -> 'Server error'
    else -> 'Unknown'
end
```

```
direction = choose (dx, dy) from
    (0, 0) -> 'stationary' -- tuple
    (0, _) -> 'vertical' -- wildcard
    (_, 0) -> 'horizontal'
    else -> 'diagonal'
end
```

```
kind = choose event from
    { type = 'click', button = 'left' } -> 'left click'
    { type = 'click' } -> 'other click' -- open table
    {} -> 'empty table'
    else -> 'other'
end
```

choose evaluates its subject once and takes the first matching arm; returns nil with no match and no else. Also a statement: 'save' -> save().

RESULT SELECTION AND PIPELINES

```
_ , content, _ = file.acRead() -- by position
content = [_*_]file.acRead() -- keep 2nd only
tail = [_*_]getValues() -- drop first
```

```
result = input |> normalise() |> validate()
maximum = getBounds() |> math.max() -- all results
maximum = getMany() |2> math.max() -- first two
```

In [mask]call(), _drops and * keeps; the final mask character governs all later positions. Pipelines insert forwarded values before arguments already written in the destination call.

SCRIPT ORGANISATION

```
import 'json' -- scripts:json.tiri
import './helpers'
import 'json' as j
```

```
namespace tools { version = '1.0' }
tools.clean = value => value.trim()
```

```
module display as mGfx -- defs + namespace
include 'xml', 'font' -- definitions only
```

```
width = tonumber(arg('width', '800'))
```

import and module are top-level declarations; CLI key=value arguments arrive as strings. mSys is the implicit Core module namespace. Wrap optional modules and their uses in @if(exists='modules:name') ... @end.

ENUMS AND STRUCTS

```
enum STATUS {
    READY = 0,
    RUNNING,
    STOPPED = 10,
}
-- STATUS_READY, STATUS_RUNNING, STATUS_STOPPED
```

```
struct Point
    X: double
    Y: double
    Label: str
end
```

```
point = struct<Point> { X=10.5, Y=20, Label='origin' }
empty = struct<Point> -- zero-initialised
```

enum and struct declarations are top-level only; enum prefixes and members use upper-case identifiers. Struct fields use storage types such as int, double, str, bool, type[N], array<type> and struct<Name>.

KOTUKU OBJECTS AND CONTEXT

```
file = obj<File> { path = 'data.txt', flags = FL_READ }
dynamic = obj.new(class_name, { name = 'worker' })
found = obj.find('main_window', 'Surface') -- or nil

err = object.acAction(arguments) -- actions ac...
err, value = object.mtMethod(arguments) -- methods mt...

with object do -- lock; always unlocks
  object.field = value
end

counter = entity { total = 0 }
function counter.add(Amount:num):num
  &total += Amount -- context field
  return &total
end
```

Most module calls and all object actions/methods return an ERR code first. Use check, checkall, a result mask or explicit handling — never discard it by accident.

SCOPED CONTEXT

```
settings = { width = 640, height = 480 }
using settings do
  &width = 800 -- settings.width
  print(&height)
end
```

using temporarily makes a table the current context. &field accesses a field on that context and && returns the context table itself.

OPERATOR PRECEDENCE — HIGH TO LOW

13	**
12	unary · · not · # · unary ~
11-9	* / % · + - << >>
8-5	& · .. · ^ ·
4	is · is not · != · < > <= >= · in · has · ≈
3-2	and · >
1	or · ?? · ?! · ? : · ?? :

Postfix ++ and ??, and suffix chains (. field, [index], (), ?.) bind tighter than binary operators. Assignment operators are statements and take no part in precedence. Parenthesise when intent is not obvious.

LAZY EXPRESSIONS AND THUNKS

```
lazy = <{ expensiveCall() }> -- inferred type
typed = <str{ loadText() }> -- explicit type
pending = isthunk(lazy)
value = resolve(lazy) -- evaluate once, cache

thunk fetchUser(Id:num):table
  return database.query(Id)
end
user = fetchUser(42) -- body stays lazy
```

A deferred expression resolves when its value is required. thunk declares a function whose call captures its arguments and returns a deferred, cached result.

STRUCT TYPE

```
point:struct<Point> = struct<Point> { X=10, Y=20 }
dynamic = struct.new(name, fields) -- runtime name
copy = struct.clone(point) -- deep copy
copy.X = 30 -- point.X stays 10

point.X = 12 -- field write
field_count = #point
has_x = 'X' in point -- field exists
exact = point is <struct Point> -- exact layout
bytes = struct.size(point) -- byte size
struct.copy(target, point) -- replace target
```

A struct may own its storage or be a live view of API or object-field memory. A borrowed view and its embedded sub-structures must not outlive their owner. Clone a value when it needs an independent lifetime.

ANNOTATIONS AND PREPROCESSING

```
@Doc(text=[[Returns a display label.]])
function label(Value:any):str
  return tostring(Value)
end

@if(debug=true)
  print('Verbose diagnostics enabled')
@end

@if(exists='modules:display')
  module display as mGfx
@end
```

Annotations apply to the function declaration immediately following them. @if(condition=value) ... @end includes or excludes source at parse time; common conditions are imported, debug, platform and exists.

Extended Doc Options

Inputs and results are matched by position, so keep their order aligned with the function signature. Use as many input and result annotations as there are parameters.

```
@Doc(text=[[
@desc Returns a display label.
@input Value to convert.
@result Display label.
@error Args: If the value is nil.
]])
function compactLabel(Value:any):str
  if Value is nil then raise ERR_Args end
  return tostring(Value)
end
```

@input and @result inherit signature type annotations, falling back to any when a type cannot be determined.